Architecture Document

for TDT4240 Project

Project/Game name: Shoot'em N Loot'em (working title)

Group number: 8

Group members: Kristian Carlenius, Fermin Martinez, Marcos Davidson, Pedro Villa & Jordan Bomolo Mia

Chosen COTS: libGDX

Primary quality attribute chosen: Modifiability

Secondary quality attribute(s): Usability, Performance

1 Introduction

1.1 Project Description

This report presents the development of a multiplayer top-down shooter designed for the Android platform. The game is implemented using Java and the LibGDX framework, and it supports real-time interaction between players through a self-hosted server. The purpose of the project is to design and implement a structured and maintainable game architecture while satisfying functional and quality-related requirements such as performance, modifiability, and usability.

The system is built around a client–server architecture in which players connect to a single shared lobby by providing only a username. Each match starts from a clean state, meaning that no progression or persistent gameplay data is carried over between sessions. The world, game rules, and player attributes are defined in a way that allows the map and gameplay parameters to be modified without restructuring the entire application. Decisions regarding networking, state management, and input handling influenced the separation of responsibilities across layers and the handling of real-time multiplayer synchronization as in multiplayer, latency is the killer of fun.

1.2 The Game Concept

The game is a competitive multiplayer top-down shooter in which all players join the same lobby and spawn into a shared arena. Upon joining, each player enters the match with a fresh character state and no prior advantages. The only requirement to participate is selecting a username, after which the player is placed into the active game world.

The world consists of a square grid-based map composed of different tile types. Each tile defines specific movement and interaction behavior. Floor tiles are fully traversable, while wall tiles block player movement and projectiles. Window or half-wall tiles prevent player movement but allow projectiles to pass through, creating strategic sightlines and defensive positions. Trap or fire tiles behave as walkable ground but inflict damage to any character standing on them. These tile types form the foundation for gameplay tactics and spatial control.

In addition to individual tiles, the map contains predefined structural layouts such as houses or a ruined church. These structures are constructed from combinations of wall, floor, and doorway tiles and are used during map generation to create varied and recognizable environments. Although the game currently uses a single map and a single shared lobby, the map design is modular and can be modified or replaced without altering the core gameplay logic.

Players control their character using on-screen touch controls. Movement is performed

using directional buttons positioned for the right thumb, allowing traversal across the grid. A virtual joystick controlled by the left thumb determines the direction the character faces and shoots. Shooting direction is therefore directly linked to the orientation defined by the joystick input. The right side of the interface also includes a button for switching between available weapons. This control scheme separates movement and aiming, enabling fluid twin-stick shooter mechanics like that of an x-box controller, adapted for mobile devices.

The objective of the game is straightforward: eliminate other players before they eliminate you. Characters have attributes including health, movement speed, and a collection of weapons. Weapons differ in damage output, projectile speed, projectile spread or size, ammunition capacity, and tier level. These attributes influence combat style and strategic choices. Items appear throughout the map inside chests, which spawn at intervals. When opened and collected, these chests provide health restoration, movement speed buffs or new weapons, introducing dynamic variation during play.

The gameplay loop is "infinite" meaning the game lobby is always running, which in turn means that you always join the active session. Dying resets you, and after a short 5 second count down respawns you at a random place on the map.

2 Architectural Drivers / Architecturally Significant Requirements (ASRs)

This section presents the architectural drivers that have influenced the design of the system. Architectural drivers are the requirements, constraints, and quality attributes that have the strongest impact on architectural decisions. For this project, the primary drivers stem from the need to support real-time multiplayer gameplay on Android devices, while maintaining a modular and modifiable codebase within a limited development timeframe.

2.1 Functional Requirements

2.1.1 Real-Time Online Multiplayer

The most influential functional driver in this project is the requirement for synchronous real-time multiplayer gameplay. All players join a single shared lobby and interact in the same game world simultaneously. Each client must continuously send player input to the server, while receiving frequent state updates containing information about other players, projectiles, and items. This requirement directly dictates the use of a client–server architecture, where the server maintains the authoritative game state and distributes updates to connected clients.

Real-time interaction imposes strict constraints on latency, update frequency, and state

synchronization. The architecture must therefore support a continuous game loop on both client and server, structured message handling, and clear separation between networking, game logic, and rendering. Without this requirement, the system could have been implemented as a purely local or asynchronous application, resulting in a significantly different architecture.

2.1.2 Dynamic Map and Tile-Based World

The game world consists of a square grid composed of multiple tile types, including floor, wall, window, and trap tiles. In addition, predefined structures such as houses and a destroyed church are constructed from combinations of these tiles during map generation. The map should be replaceable or modifiable without changing the core engine logic.

This requirement influences the architecture by enforcing a separation between game logic and map definition. Tile behavior must be interpreted through clearly defined rules rather than hard-coded behavior tied to specific maps. The architecture must therefore support external map definitions and flexible loading mechanisms to ensure that future changes to level design do not require structural rewrites.

2.1.3 Weapon and Item System

Weapons in the game differ in damage, projectile speed, projectile spread or size, ammunition count, and tier level. Items spawn in chests throughout the map and may provide either health or new weapons. Since players start each match from an equal baseline, the weapon and item system is central to gameplay progression within a single session.

To support balancing and iteration, the architecture must allow weapon attributes and item behavior to be adjusted without affecting unrelated systems. The design must therefore separate weapon data, combat logic, and rendering concerns. This enables modifications to weapon statistics or the addition of new item types without structural changes to networking or core game flow.

2.1.4 Touch-Based Control Scheme

The game is controlled using on-screen touch input, where directional buttons handle movement and a virtual joystick controls facing and shooting direction. Additionally, a button allows players to switch weapons during gameplay. This input scheme requires low-latency response and clear separation between input processing and character behavior.

Because mobile input differs significantly from keyboard or controller input, the architecture must isolate input interpretation from domain logic. This ensures that changes to control layout or sensitivity do not affect movement rules or combat systems. Supporting responsive and intuitive controls is essential for maintaining gameplay quality.

2.2 Quality Attributes

2.2.1 Modifiability

Modifiability is the primary quality attribute guiding architectural decisions. The project must allow adjustments to map layout, weapon statistics, tile behavior, and gameplay rules without widespread code changes. This requirement strongly motivates the use of layered architecture and clear separation of responsibilities between domain logic, presentation, and networking.

By isolating game state management from rendering and input handling, the system supports incremental development and easier debugging. Modifiability is particularly important given the experimental nature of balancing weapons and tuning gameplay mechanics during development.

2.2.2 Usability

Usability is a secondary quality attribute, focusing on intuitive controls and clarity of gameplay. Since players only need to enter a username to join the match, the onboarding process must be simple and direct. The control scheme must be responsive and easy to understand, and the user interface must clearly communicate health, weapon status, and available actions.

Architecturally, this requires careful separation between user interface rendering and game logic so that adjustments to button placement, scaling, or feedback mechanisms can be made without affecting core systems. The design must also ensure consistent behavior across different Android devices and screen sizes.

2.2.3 Performance

Performance is another critical quality attribute due to the real-time multiplayer nature of the game. The client must render gameplay smoothly while processing network updates, and the server must handle input messages and distribute state snapshots at a stable tick rate. The architecture must therefore minimize unnecessary object creation, reduce rendering overhead, and structure networking code efficiently.

The system should maintain a stable frame rate on mobile devices while supporting frequent position updates to preserve gameplay responsiveness. If performance were not a concern, networking and rendering logic could be implemented more loosely; however, the multiplayer requirement makes performance architecturally significant.

2.3 Business Requirements

A key business constraint is the limited development period. The project must be completed within the 10 weeks given by the course, which necessitates prioritizing a functional minimum viable product before extending features. Architectural decisions therefore emphasize clarity, maintainability, and incremental implementation rather than large-scale experimental designs.

The game is self-hosted, meaning the multiplayer server runs on infrastructure controlled by the developers. This avoids reliance on external backend providers but places responsibility for availability and deployment on the project team. Since the application is distributed outside of official app stores, it must be packaged and signed for direct installation. These constraints simplify backend dependencies but require deliberate architectural planning to ensure the system remains stable and demonstrable within the allotted time.

Finally, the architecture must be sufficiently documented and structured to allow external evaluators to understand its components, relationships, and design rationale. Clear separation of layers and explicit responsibility boundaries support both maintainability and evaluability, which are essential within an academic setting.

3 Stakeholders and Concerns

Stakeholder	**Concerns**
Project Team	The team aims for a quick and efficient implementation of the game. As such, modifiability is a key concern, in order to quickly be able to implement new weapon types, upgrades, map elements, tiles, etc.
Teaching Staff	The main concern for the teaching staff is that the team acquire a good understanding of key aspects of software architecture, including use and implementation of architectural tactics and patterns. As such, it is important for the documentation of the project to be as clear, comprehensive and accurate as possible.
End Users	The user wishes to have an enjoyable experience playing this game. As such, key concerns will include usability, as it will be important for the system to be as intuitive as possible, and performance, as the user will want the system to be fast, responsive and stable.
ATAM Evaluators	The evaluators will want the evaluation of the project to be as straightforward as possible. As such, it is important for the usage of architectural patterns and tactics to be simple to understand and for the documentation of the project to be clear and concise.

4 Architectural Viewpoints

View	Purpose	Stakeholders	Notation
Logical View	Describes system functionality and interactions between classes.	Developers, Teaching Staff, End Users, ATAM Evaluators.	UML Class Diagram
Process View	Describes the system's runtime behaviour, including processes and workflow.	Developers, Teaching Staff, ATAM Evaluators.	UML Activity Diagram, UML Sequence Diagram
Development View	Describes system components such as packages, libraries and sub-systems.	Developers, Teaching Staff, ATAM Evaluators.	Package Diagram
Physical View	Describes how software components map to physical hardware.	Developers, Teaching Staff, ATAM Evaluators.	UML Deployment Diagram

5 Architectural Tactics

This section describes the specific architectural tactics applied to achieve the project's primary quality attributes: modifiability, usability, and performance. The tactics are expressed explicitly and mapped to how they are realized in the game.

5.1 Modifiability Tactics

Modifiability tactics are used to reduce the cost, complexity, and risk associated with making changes to the system.

Increase Cohesion

Split Modules: The codebase is divided according to layered architecture and MVC. The client is separated into presentation (screens, renderers, input), application (game flow

coordination), domain (game rules and entities), and infrastructure (networking and content loading). Within gameplay, responsibilities such as combat logic, tile interaction, and map generation are placed in distinct classes. This ensures that modules contain closely related responsibilities, making the system easier to understand and modify.

Separate Data from Behavior Configuration: Weapon attributes, map definitions, and tile configurations are defined externally (JSON-based content definitions). By separating configuration data from gameplay logic, changes to balance values or map layouts do not require structural code modifications.

Isolate Map Generation Logic: Prebuilt structures such as houses or churches are encapsulated as reusable structure definitions rather than embedded directly into the main game logic. This allows changes to map composition without affecting combat or movement systems.

Reduce Coupling

Introduce Explicit Interfaces: Communication between layers is performed through defined interfaces, such as a network client interface between domain logic and infrastructure. This ensures that changes in networking implementation do not propagate into gameplay rules or rendering code.

Use Intermediary Controllers: In the MVC structure, controllers act as intermediaries between views and models. Input events are processed by controllers rather than directly modifying domain objects from UI components. This prevents direct dependencies between rendering code and gameplay logic.

Restrict Dependencies Across Layers: Lower layers do not depend on higher layers. Domain logic does not reference LibGDX classes directly, and presentation components do not directly modify internal game state without passing through controllers. This directional dependency rule prevents ripple effects when changes are introduced.

5.2 Usability Tactics

Usability tactics are divided into supporting user initiative and supporting system initiative.

Support User Initiative

Provide Immediate Feedback: The system visually reflects player actions such as movement, shooting direction, weapon switching, and damage taken. This ensures that user input is acknowledged instantly, reducing uncertainty during gameplay.

Enable Cancel and Back Navigation: Outside active gameplay, users can cancel connection

attempts, return from the game lobby to the main menu, or change their username before joining. This prevents users from becoming locked into unintended states.

Persistent User Preferences: Basic settings such as sound toggling and joy-stick placement are stored and restored between sessions. This allows players to customize aspects of the experience without altering gameplay mechanics.

Support System Initiative

Maintain System State Model: The application explicitly tracks its current state (menu, connecting, in-game). The interface only exposes actions relevant to the current state, preventing invalid operations such as attempting to shoot before spawning.

Connection Monitoring: The client continuously monitors its connection to the self-hosted server. If the connection is lost, the system notifies the user and prevents further gameplay input until reconnection or exit. This ensures transparency and predictable behavior in network failure scenarios.

5.3 Performance Tactics

Performance tactics ensure that the game meets timing requirements for both rendering and multiplayer synchronization.

Control Resource Demand

Fixed Tick Rate: The server updates the game world at a fixed tick rate, and clients process updates at a controlled frequency. This prevents unbounded message handling and stabilizes CPU usage.

Prioritize Gameplay Events: Critical events such as movement updates and damage resolution are processed before secondary tasks such as visual effects. This ensures that core gameplay remains responsive even under load.

Limit Network Payload Size: Instead of transmitting full world data on every update, the server sends compact snapshots containing only necessary state information. This reduces bandwidth consumption and processing overhead.

Reduce Computational Overhead

Move Calculations Outside Render Loop: Expensive computations, such as collision resolution and state updates, are handled in update methods rather than directly inside

the rendering routine. Since the render method executes every frame, this prevents unnecessary workload spikes.

Reuse Objects: Frequently created objects such as projectiles or temporary vectors are reused where possible to reduce garbage collection pressure in the Java runtime on Android devices.

Introduce Concurrency

Asynchronous Networking: Network communication is handled on separate threads from the rendering loop. This ensures that delays in receiving data do not block frame updates or input responsiveness.

Decouple Server I/O from Simulation Loop: On the server side, connection handling and message reception are separated from the world simulation tick. This prevents slow clients from affecting the timing consistency of the entire match.

6 Design and Architectural Patterns

This section describes the architectural patterns and design patterns used in the project. The architecture is driven by the need to support a real-time multiplayer mobile game while keeping the system easy to evolve during development. For that reason, we explicitly structure the solution around a client–server architecture, layered separation of responsibilities, and Model-View-Controller (MVC) within the client. In addition, a small set of design patterns is applied to keep object creation, state transitions, and event flow manageable as the project grows.

6.1 Architectural Patterns

6.1.1 Client-Server

The system is implemented using a client–server pattern in order to support simultaneous gameplay between multiple players connected to the same match. The Android application acts as the client and is responsible for rendering, collecting input, and presenting the game state to the player, while the self-hosted server is responsible for coordinating a shared world state and distributing updates to all connected clients. This separation supports modifiability because client-side features such as UI layout, rendering, and control tuning can be changed without restructuring the server, while server-side features such as tick rate, state synchronization, and spawn logic can be evolved without rewriting the client. At the same time, the client–server pattern introduces performance-related

constraints since gameplay responsiveness depends on network conditions and on how efficiently messages are processed. The architecture therefore requires structured message handling and a stable update rhythm so that network delays do not dominate the user experience.

6.1.2 Layered Architecture

The client is organized using a layered architecture to limit coupling between concerns that change for different reasons. Presentation code handles LibGDX screens, rendering, and touch controls, while application-level code coordinates game flow such as joining the lobby, starting a session, and applying server updates. The domain layer contains the core rules and data structures describing players, weapons, projectiles, tiles, and interactions such as damage and collisions. Finally, the data/infrastructure layer contains networking adapters, asset loading, and file-based content loading for maps and weapon definitions. This separation supports modifiability by ensuring that changes to map formats or weapon balancing do not force changes in rendering code, and that networking implementation details do not leak into gameplay rules. A layered structure also improves maintainability and testability because rules can be reasoned about without requiring the full LibGDX runtime environment.

6.1.3 Model–View–Controller

On the client side, the project applies the Model–View–Controller pattern to keep game state and rules separate from input and rendering. The model corresponds to the domain state representing players, weapons, tiles, and ongoing match state. The view corresponds to LibGDX screens and renderers that draw the world and HUD elements and display joystick and button controls. The controller interprets touch input and translates it into game actions such as movement intent, facing direction, shooting, and weapon switching, and it also triggers the appropriate networking messages to the server. In practice, LibGDX's screen lifecycle encourages some blending of view and controller responsibilities, since screens must respond to lifecycle callbacks and often act as integration points for input routing. The architecture addresses this by keeping the majority of control decisions in controller classes, while screens primarily act as the composition point that connects views, controllers, and the current model. This organization supports usability because changes to the interface and control layout can be made without altering core gameplay rules, and it supports modifiability because each responsibility is updated in isolation during iteration.

6.2 Design Patterns

6.2.1 Creational Patterns

A controlled single-instance approach is used for central runtime services such as the current game session, shared asset manager, and network client. These objects represent global resources that should not be duplicated during execution because multiple instances would lead to inconsistent state, duplicated resource loading, or conflicting network connections. This is implemented as a singleton-style service in practice, either through explicit single-instance classes or through a dependency injection module that constructs and stores one instance per service for the duration of the application.

A factory-based approach is used where object creation should be centralized and decoupled from the rest of the system. This applies particularly to screen creation and to spawning domain objects such as projectiles, pickups, and chest contents. A screen factory allows the navigation system to request a new screen without the caller needing to know which renderers, controllers, and services must be injected. Similarly, entity factories allow the gameplay logic to spawn objects using data-driven definitions, which supports modifiability because new weapon tiers or pickup types can be introduced without scattering construction logic throughout the codebase.

6.2.2 Behavioral Patterns

Event-driven communication is used to propagate changes from systems that produce updates to systems that consume them, without creating tight coupling. This pattern is especially relevant on the client where user input events, network events, and gameplay events all occur concurrently. For example, the networking layer receives snapshot updates from the server and emits them as events to the application layer, which then updates the model and triggers view refresh. This supports modifiability because the producer of events does not need to know which specific systems react to them, making it easier to add new reactions such as sound effects, UI indicators, or debug overlays without restructuring the networking code.

State-based behavior is used to manage the application's discrete modes, such as menu, lobby, and active gameplay. LibGDX screens naturally support this approach because only one screen is active at a time, and each screen defines what input is handled and what information is shown. Modeling the application as explicit states reduces accidental cross-state behavior, improves usability by ensuring the correct UI is presented at the correct time, and supports modifiability by allowing new screens or states to be added with limited impact on existing ones.

6.2.3 Structural Patterns

A façade-style approach is used to provide a simplified interface over complex subsystems. In practice, this is represented by higher-level services such as a game session service that exposes simple operations like connect, join, start match, and disconnect while hiding details about message encoding, connection management, and tick timing. This reduces the number of dependencies that presentation components must understand and helps keep controllers focused on gameplay intent rather than infrastructure details. This contributes primarily to modifiability and understandability, as the internals of networking or asset handling can be changed behind the façade without affecting the rest of the client.

7 Architectural Views

7.1 Logical View

The Logical View illustrates the static structure of the system, detailing the core classes, interfaces, and their relationships. Given that Modifiability is the primary quality attribute of this project, presenting the system as a single monolithic class diagram obscures the clear architectural boundaries we have established.

To effectively demonstrate the separation of concerns and how the architecture supports future modifications, the logical view has been decomposed into four distinct diagrams: a high-level package overview, followed by detailed views of the Model, Controller, and View layers. This approach explicitly shows how the Model remains independent of rendering and networking concerns, and how the View enforces a strict read-only relationship with the game state.

7.1.1 High-Level Package Overview

The system is organized into five primary packages based on the Model-View-Controller (MVC) architectural pattern, supplemented by dedicated data packages for client and server configurations. The macro-architecture strictly dictates dependency directions: the View reads from the Model and forwards user intents to the Controller, while the Controller acts as the authoritative mutator of the Model.

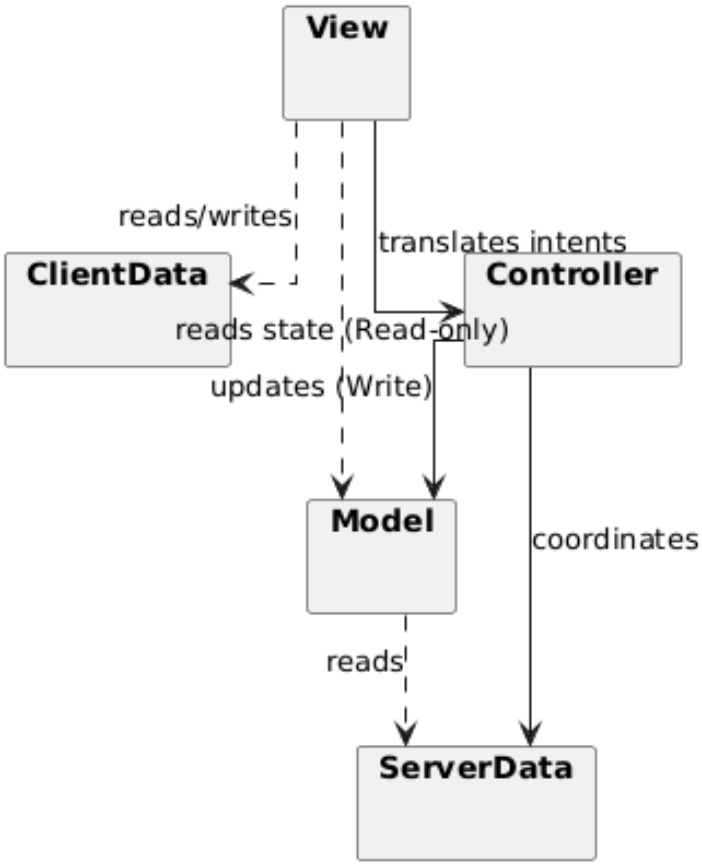

Figure 1: High-Level Package Overview illustrating architectural layers and dependencies.

7.1.2 Model and Data Layer

The Model layer encapsulates the core domain logic, game state, and entity definitions. It is entirely decoupled from the LibGDX rendering engine and network transport layers, ensuring that business rules can be tested and modified in isolation. The `GameModel` orchestrates the `WorldState`, which aggregates all active entities such as `Player`, `Projectile`, and `Weapon`.

Furthermore, this layer manages the ingestion of server-side data via the `ServerData` package. The `ContentLoader` parses JSON definitions (e.g., `WeaponDef`, `MapDefinition`), which the `MapGenerator` and `EntityFactory` use to instantiate the game world. This data-driven approach directly supports modifiability, allowing game balance, weapon stats, and map layouts to be adjusted without recompiling the core application logic.

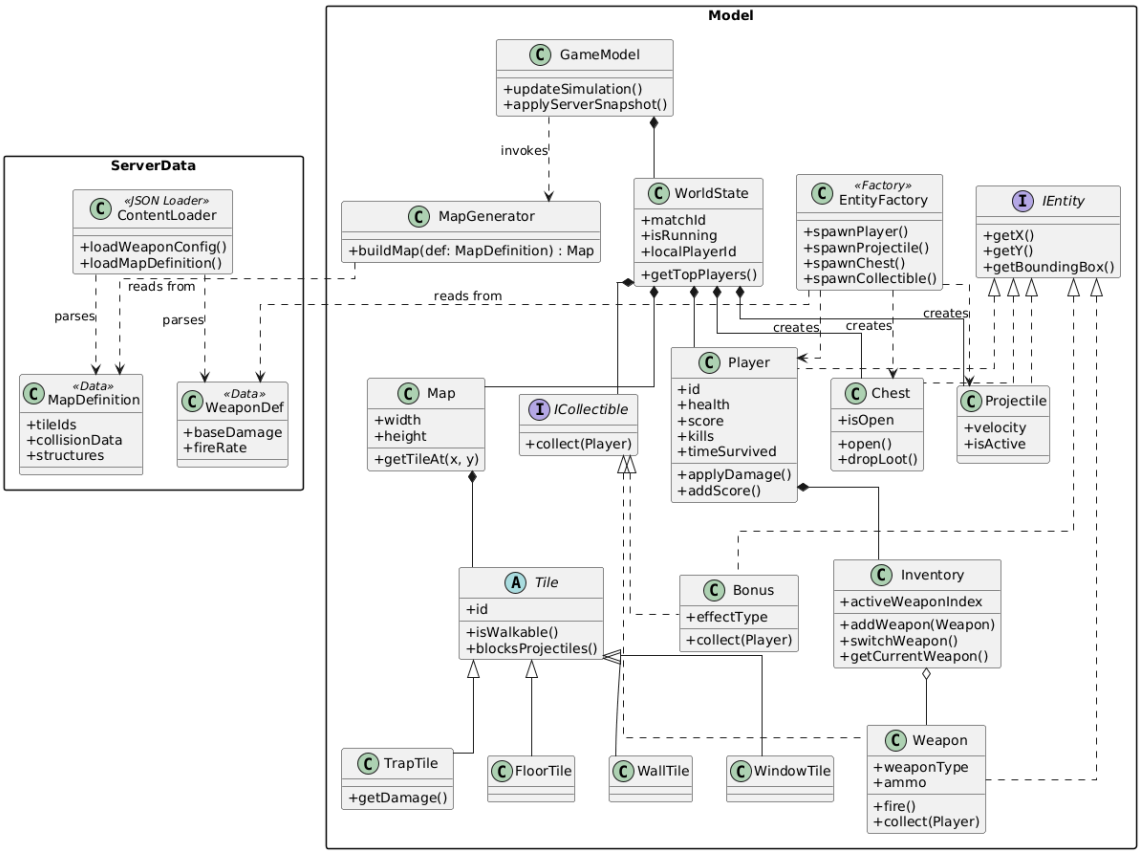

Figure 2: Model and Data Layer demonstrating entity aggregation and data-driven instantiation.

7.1.3 Controller and Network Layer

The Controller layer acts as the intermediary between the user interfaces, the network subsystem, and the game state. It handles session management, multiplayer routing, and the application of server snapshots to the local game state.

To simplify the complexities of networking, the `GameSessionService` serves as a *Facade*, providing a clean interface for the UI to connect to lobbies. When the `AsyncNetworkClient` receives a new snapshot from the server, the `GameController` processes this data and safely applies the updates to the `GameModel`. By isolating the network logic behind abstract interfaces (`INetworkListener`, `INetworkClient`), the underlying networking implementation (e.g., swapping between TCP and UDP) can be modified without affecting the rest of the application.

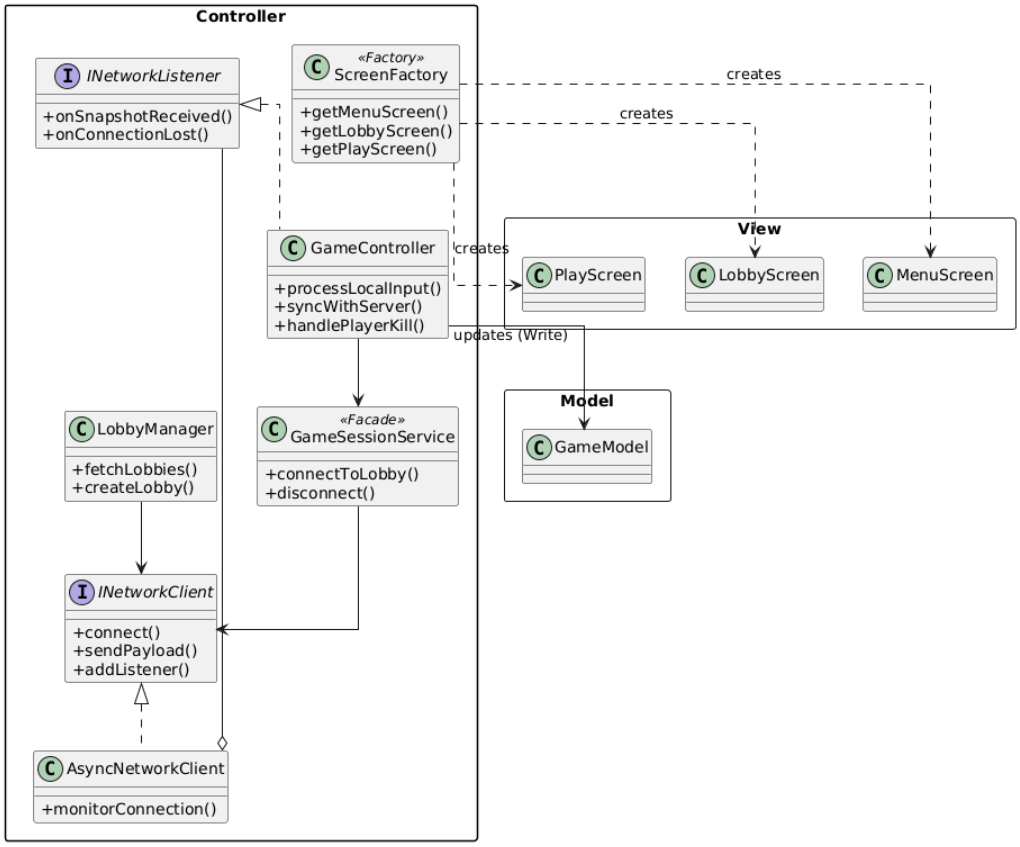

Figure 3: Controller and Network Layer showing session management and network event routing.

7.1.4 View and UI Layer

The View layer handles all visual representation and user input capture, relying on the LibGDX framework. It is organized around a `BaseScreen` architecture, managed by a `ScreenFactory` to handle transitions between the `MenuScreen`, `LobbyScreen`, and `PlayScreen`.

A critical architectural decision in this layer is the strict enforcement of the read-only rule. Classes like `WorldRenderer` and `HudRenderer` only query the `WorldState` and `Player` models for rendering purposes; they never modify them. User actions captured by the `InputHandler` (such as joystick movement or screen tapping) are translated into intents and forwarded to the `GameController`. Additionally, the View layer interacts with the `ClientData` package to load persistent user preferences and retrieve graphical and audio assets from the `AssetStore`.

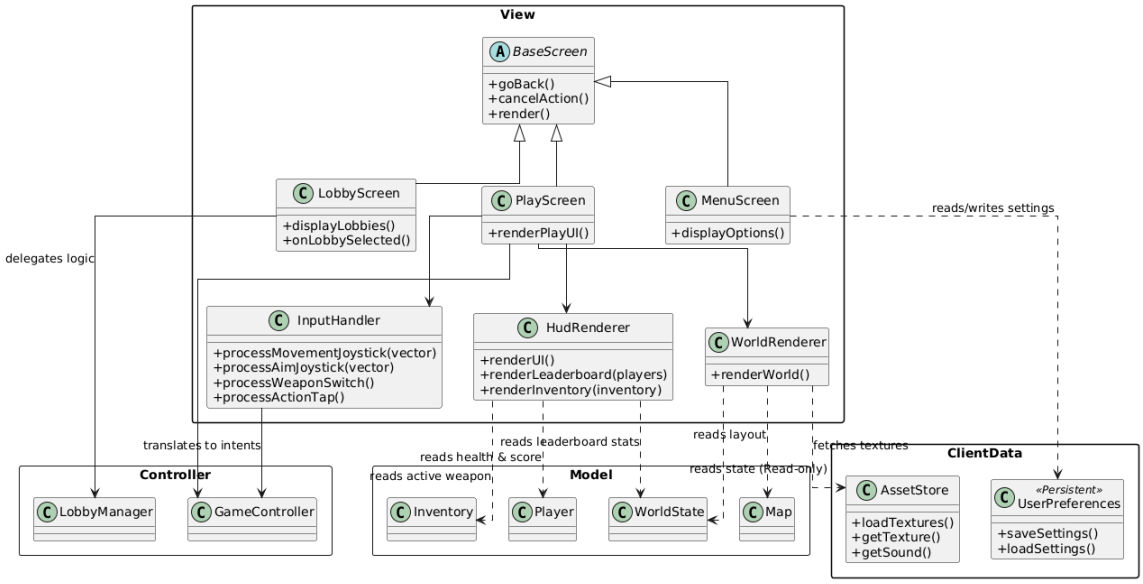

Figure 4: View and UI Layer highlighting strict read-only dependencies on the Model.

7.2 Process View

The process view describes the runtime behavior of the system, including how processes communicate, how user actions flow through the application, and how the client and server interact during gameplay. This view is particularly relevant given the real-time multiplayer nature of the game, where continuous synchronization between clients and the server defines the core gameplay experience.

7.2.1 Application Flow

The activity diagram in Figure 5. presents a high-level overview of the application's process flow from the user's perspective.
When the application is first launched, the system checks whether it is the user's first session. If so, the user is directed to a tutorial screen that introduces the game controls and rules. Otherwise, the user proceeds directly to the main menu. From the main menu, the user enters a username and requests to join the active lobby. The system attempts to connect to the self-hosted server. If the connection fails, the user is notified and returned to the main menu. Upon successful connection, the player enters the active game session and begins gameplay immediately.

During gameplay, the player may be eliminated when their health reaches zero. In this case, a death overlay is displayed while a five-second countdown timer runs. After the countdown completes, the player is automatically respawned at a random position on the map with reset health and default weapon. This death–respawn cycle continues indefinitely as part of the persistent gameplay loop. The player may leave the session at any time, which disconnects them from the server and returns them to the main menu.

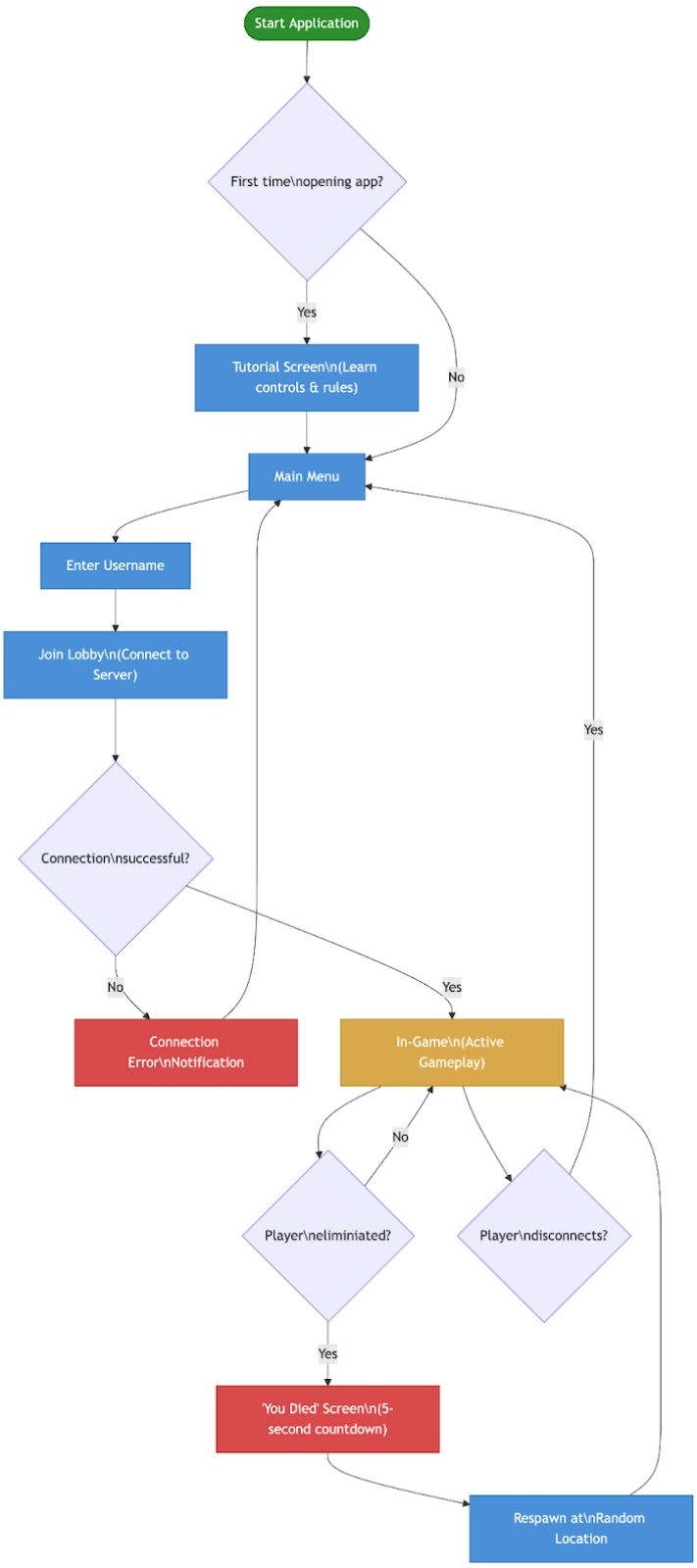

Figure 5: Game program flow chart.

7.2.2 Client–Server Interaction

The sequence diagram in Figure 6. shows how the client and server communicate during a gameplay session. The interaction is split into three phases: connection, gameplay loop, and death/respawn.

Connection phase. The client sends the chosen username to the server. The server registers the player and replies with the current world state (map, players, projectiles, and items. The client renders the game world from this initial snapshot).

Gameplay loop. The client captures touch input (movement, aim, shoot, weapon switch) and sends it to the server as lightweight input messages. The server runs at a fixed tick rate: each tick, it processes all inputs, moves players and projectiles, resolves collisions and damage, handles chest spawns and pickups, and updates scores. It then sends a compact state snapshot back to each client. The client applies the snapshot and renders the world, HUD (health, weapon, ammo), and the leaderboard overlay.

Death and respawn. When a player's health hits zero, the server sends a kill event to the eliminated player and notifies all others. The dead player sees a "You Died" overlay with a 5-second countdown. During this time, the client still receives updates but stops sending input. After the countdown, the client requests a respawn. The server picks a random valid position, resets health and weapon, and confirms. The player resumes from there.

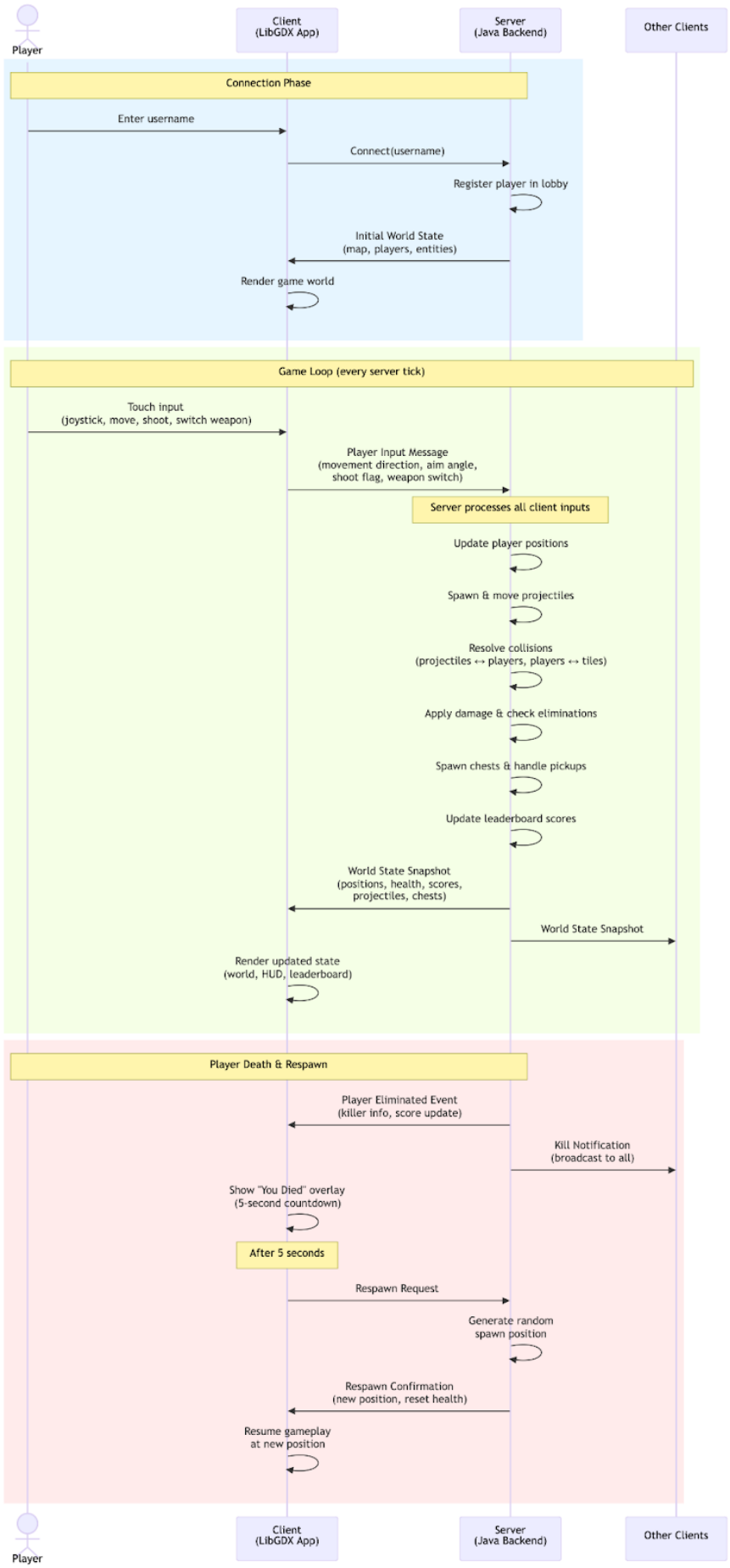

Figure 6: Cleient-Server interaction diagram.

7.3 Deployment View

The Development View describes the repository-level structure of the system and how responsibilities are realized in code to support parallel development, clear boundaries, and controlled integration. The project is divided into three primary modules: a client module (Android, LibGDX), a standalone Java server module, and a contract module defining the shared communication protocol.

The client module contains presentation, input handling, and client-side coordination logic. It follows a Model–View–Controller structure, with separate packages for view (screens and rendering), controller (game flow and input interpretation), and model (client-side game state). Networking is isolated in a dedicated `net` package, and configuration and resources are encapsulated in a `data` package. This separation enables independent evolution of interface, gameplay logic, and networking.

The server module is organized by runtime responsibility. A `net` package manages connections and message handling, a `simulation` package runs the fixed-rate game loop and updates the authoritative world state, a `model` package represents server-side state, and a `snapshot` package distributes world updates to clients. This structure isolates networking, simulation, and state management to support incremental changes.

The contract module defines shared message structures and serialization formats used by both client and server networking layers. Centralizing protocol definitions ensures consistency and controlled evolution of communication.

Figure 7. illustrates the module decomposition and primary dependencies. Only the networking layers depend on the contract module, while higher-level components remain independent of transport and serialization concerns, reinforcing low coupling and high cohesion.

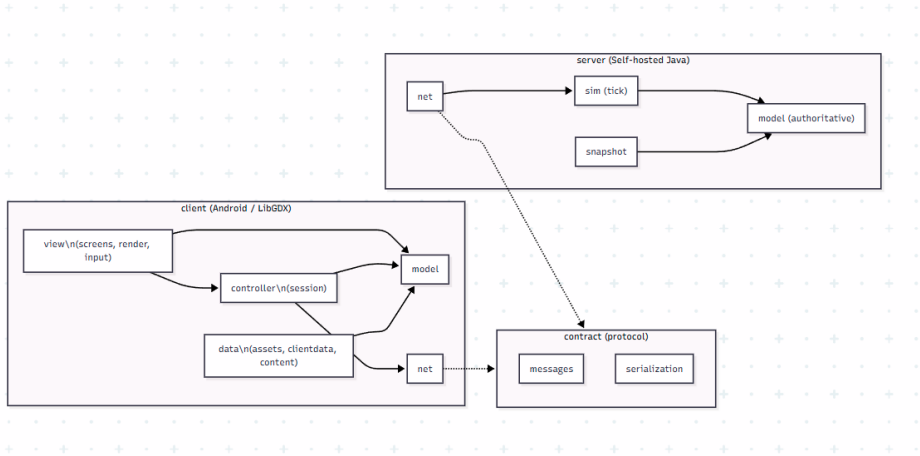

Figure 7: Cleient-Server interaction diagram.

7.4 Physical View

The Physical View illustrates how our software components map directly to the physical hardware that runs them. Because our project is a real-time multiplayer top-down shooter, this view highlights the distributed network setup that bridges the players' mobile devices and our central server.

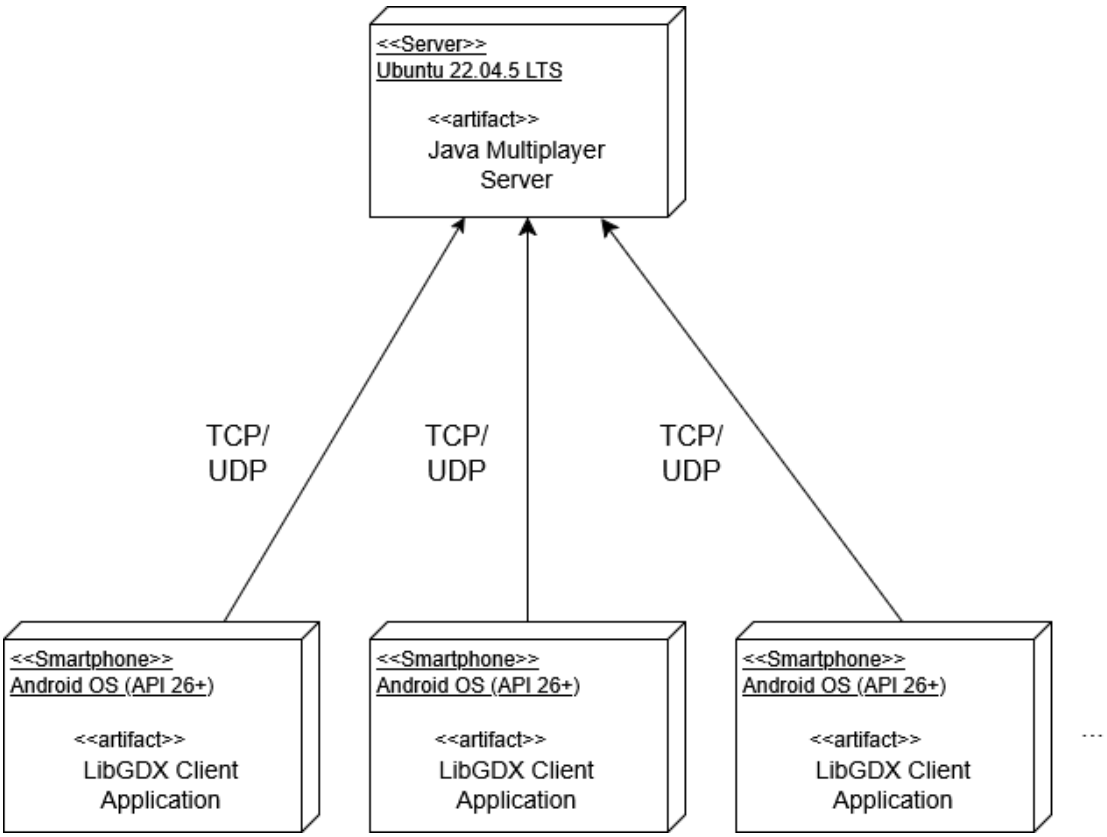

Figure 8: UML Deployment Diagram showing the client-server topology and execution nodes.

7.4.1 Deployment Nodes and Artifacts

Our architecture follows a strict client-server topology, divided into client (`«Client»`) and server (`«Server»`) nodes.

The heart of the game is a self-hosted Ubuntu 22.04.5 LTS machine running the `Java Multiplayer Server` artifact. This server acts as the absolute authority on the game state, taking charge of match coordination, physics resolution, and lobby management.

Centralizing the core game logic here heavily supports our primary quality attribute of Modifiability. It allows us to seamlessly update server-side mechanics, tweak map layouts, or balance weapons without ever forcing players to download a new client update.

Players connect using Android devices (targeting API 26+ to ensure modern Android and LibGDX compatibility) that host our `LibGDX Client Application` artifact. In the context of game logic, the client functions as a lightweight "dumb terminal." Its main responsibilities are simply rendering the visual state provided by the server and capturing the player's touch inputs to send back over the network.

7.4.2 Communication Protocols

To keep gameplay smooth and meet our Performance goals, the communication layer between the clients and the server is designed to balance reliability with real-time speed. The system uses a dual-protocol approach, employing both TCP and UDP.

We rely on TCP, state-changing events where dropping a packet is unacceptable. This includes essential steps like initial authentication (submitting a username), joining the shared lobby, and loading static map data.

On the other hand, we use UDP for the high-frequency, real-time synchronization of the active game loop. By relying on UDP for rapid updates—such as player movement vectors, projectile tracking, and server state snapshots—we minimize network overhead and keep latency as low as possible for a responsive 2D shooter experience.

8 Consistency among Architectural Views

The logical, process, physical, and deployment views describe the same architectural structure from different perspectives and are mutually consistent in terms of responsibilities, communication paths, and runtime behavior. In the logical view, the client is structured according to layered architecture and MVC, where the View (LibGDX screens and renderers), Controller (input handling and coordination), and Model (game state, entities, and rules) are clearly separated. This separation is directly reflected in the process view, where the client runtime is divided into a rendering loop, an input-processing flow, and an asynchronous networking component. The render loop continuously reads from the model to draw the world, while controller logic processes local input and forwards intent to the server. Network updates from the server are received in a separate execution context and applied to the model in a controlled manner, ensuring that rendering and simulation remain synchronized without tight coupling between threads.

The server-side process view is also consistent with the logical model. The server maintains the authoritative world state and executes a fixed-rate simulation loop responsible for

processing player inputs, resolving combat interactions, updating entity positions, and spawning items. Connection handling and message reception occur independently of the simulation tick, preventing slow clients from blocking world updates. This aligns with the logical separation between infrastructure concerns (network transport) and domain logic (world state and rules). The single shared lobby and single active map described in the logical view are therefore realized at runtime as one authoritative server instance coordinating all connected clients.

The deployment view reinforces this consistency by mapping logical components to physical nodes. Each Android device deploys one instance of the LibGDX client application, which contains all presentation, control, and client-side logic. The multiplayer server is deployed as a separate Java application running on a self-hosted Ubuntu machine. Communication between nodes occurs over TCP/UDP, as illustrated in the physical view, directly corresponding to the logical networking interfaces and asynchronous processes defined in the process view. No additional runtime components are introduced in deployment beyond those already identified logically, ensuring alignment across all architectural views.

9 Architectural Rationale

The chosen architecture is the result of balancing real-time multiplayer requirements, modifiability goals, and the practical constraints of a mobile school project. The decision to adopt a client–server architecture was driven primarily by the need for synchronized gameplay in a shared world. Since all players join the same lobby and interact in the same map simultaneously, a centralized authority is necessary to maintain a consistent world state. A self-hosted Java server provides full control over simulation timing, message formats, and update frequency without introducing dependencies on external backend services. This choice simplifies debugging, supports predictable performance, and aligns with the limited scope of the project, where scalability and enterprise-grade infrastructure are not primary concerns.

On the client side, the combination of layered architecture and Model–View–Controller was selected to support modifiability and clarity. The game is expected to undergo balancing adjustments, map modifications, and UI refinements during development. By separating domain logic from rendering and input processing, changes to one concern can be made with minimal impact on others. The layered approach further strengthens this separation by isolating infrastructure concerns, such as networking and asset loading, from core gameplay rules. This structure reduces coupling, clarifies responsibilities, and improves maintainability within a collaborative academic setting.

LibGDX was chosen as the underlying framework because it provides a stable rendering loop, cross-device compatibility on Android, and efficient handling of graphics and input without requiring low-level OpenGL management. Although LibGDX screens can blur strict MVC boundaries, the architecture compensates by centralizing control logic in dedicated controllers and maintaining a clear domain model independent of rendering classes. This pragmatic compromise preserves architectural clarity while leveraging

framework capabilities.

Finally, the overall design reflects a deliberate trade-off between robustness and complexity. Advanced concerns such as large-scale scalability, extensive security validation, and distributed microservices were intentionally avoided to maintain feasibility within the project timeline. Instead, the architecture prioritizes clear separation of concerns, predictable runtime behavior, and sufficient performance for real-time multiplayer gameplay on mobile devices. This rationale ensures that the system remains coherent, extensible, and demonstrable while staying aligned with the project's academic objectives.

10 Issues (Optional)

Won't have much to say at this point.

11 Changes

Date	**Change**	**Comment**
15.02.2026	Created initial version of the document.	None.
23.02.2026	Last update before delivery.	None.